Lista de Exercícios: Classe abstrata

Prof. Gabriel Rodrigues Caldas de Aquino

Compilado em:
September 9, 2026

1 Itens de Empréstimo

Crie um sistema para gerenciar itens que podem ser emprestados em uma biblioteca.

1. Crie uma **classe abstrata** chamada `Item`. Ela deve ter os seguintes atributos:

   - `titulo` – título do item,
   - `nomePessoa` – nome da pessoa que pegou emprestado,
   - `dataEmprestimo` – data em que o item foi emprestado.

   Além disso, a classe deve possuir um **método abstrato** chamado `exibirDetalhes()`, que será implementado pelas classes filhas para mostrar informações específicas de cada tipo de item.

2. Crie duas classes concretas que herdem de `Item`:

   (a) `Livro`: deve conter os atributos:
   - `autor`,
   - `numPaginas` – número de páginas do livro.

   (b) `CD`: deve conter os atributos:
   - `artista`,
   - `numFaixas` – quantidade de faixas do CD.

3. Crie pelo menos **3 livros** e **3 CDs**, cada um emprestado para pessoas diferentes, e coloque todos os itens em uma única lista.

4. Percorra essa lista e imprima, para cada item:

   - o tipo do item,
   - o título,
   - o nome da pessoa que pegou emprestado,
   - a data do empréstimo,
   - e os atributos específicos do item (como autor e número de páginas, ou artista e quantidade de faixas).

2 Formas Geométricas

Crie um sistema para representar formas geométricas utilizando herança e polimorfismo.

1. Crie uma **classe abstrata** chamada `Forma`. Ela deve ter os seguintes atributos:

   - `nome` – nome da forma geométrica,
   - `cor` – cor da forma,
   - `espessuraBorda` – espessura da borda,
   - `dataCriacao` – data em que a forma foi criada.

   Além disso, a classe deve possuir um **método abstrato** chamado `exibirDetalhes()`, que será implementado pelas classes filhas para mostrar informações específicas de cada forma.

2. Crie duas classes concretas que herdem de `Forma`:

   (a) `Retangulo`: deve conter os atributos:
   - `largura`,
   - `altura`.

   Também deve possuir:
   - Um método para calcular a **área**: largura $\times$ altura,
   - Um método para calcular o **perímetro**: $2 \times$ (largura + altura).

   (b) `Circulo`: deve conter o atributo:
   - `raio`.

   Também deve possuir:
   - Um método para calcular a **área**: $\pi \times \text{raio}^2$,
   - Um método para calcular o **perímetro**: $2 \times \pi \times$ raio.

3. Crie diferentes formas geométricas (pelo menos 2 retângulos e 2 círculos) e armazene todas em uma única lista.

4. Percorra a lista e utilize o método `exibirDetalhes()` para mostrar, para cada forma:

   - nome,
   - cor,
   - espessura da borda,
   - data de criação,
   - tipo da forma,
   - medidas específicas (largura/altura ou raio),
   - área e perímetro calculados.

5. Sinta-se livre para adicionar outros tipos de formas, se desejar.

3 Animais

Crie um sistema para representar diferentes tipos de animais utilizando herança e polimorfismo.

1. Crie uma **classe abstrata** chamada `Animal`. Ela deve ter os seguintes atributos:

   - `nome` – nome do animal,
   - `idade` – idade do animal.

   A classe deve possuir:

   - Um **método abstrato** chamado `emitir_som()`, que será implementado pelas subclasses para exibir o som específico de cada animal.
   - Um método `apresentar()` que imprime o nome e a idade do animal.

2. Crie três subclasses que herdem de `Animal`:

   (a) `Cachorro` – o método `emitir_som()` deve imprimir `"auau"`.

   (b) `Gato` – o método `emitir_som()` deve imprimir `"miau"`.

   (c) `Pato` – o método `emitir_som()` deve imprimir `"quack"`.

   Além disso, cada subclasse deve sobrescrever o método `apresentar()` para incluir uma mensagem personalizada, além do nome e idade do animal.

3. Crie uma lista contendo pelo menos **uma instância de cada tipo de animal**.

4. Percorra a lista e, para cada animal, chame:

   - O método `emitir_som()` para exibir o som específico.
   - O método `apresentar()` para mostrar as informações detalhadas.

5. Demonstre o uso de **polimorfismo**, mostrando que, mesmo com uma lista genérica de `Animal`, cada subclasse executa seu comportamento específico.

4 Veículos e Oficina

Crie um sistema para gerenciar veículos em uma oficina mecânica, utilizando herança e polimorfismo.

1. Crie uma **classe abstrata** chamada `Veiculo`. Ela deve ter os seguintes atributos:

   - `marca` – marca do veículo,
   - `modelo` – modelo do veículo,

- `ano` – ano de fabricação,
- `dataEntrada` – data em que o veículo deu entrada na oficina.

A classe deve possuir:

- Um **método abstrato** chamado `calcularOrcamento()`, que retorna o valor estimado do conserto.
- Um método `exibirDetalhes()` que mostra as informações básicas do veículo.

2. Crie as seguintes subclasses que herdem de `Veiculo`:

   (a) `Carro`:
   - Deve ter um atributo `numPortas`.
   - O método `calcularOrcamento()` deve retornar um valor baseado no ano do carro e na quantidade de portas, considerando que carros mais antigos e com mais portas têm custo de manutenção maior.

   (b) `Moto`:
   - Deve ter um atributo `cilindrada`.
   - O método `calcularOrcamento()` deve retornar um valor baseado na cilindrada, sendo que motos de maior cilindrada têm custo de manutenção mais alto.

3. Crie uma lista contendo pelo menos **3 veículos de tipos variados** (por exemplo, 2 carros e 1 moto).

4. Percorra a lista e, para cada veículo, chame:
   - O método `exibirDetalhes()` para mostrar as informações básicas,
   - O método `calcularOrcamento()` para mostrar o valor estimado do conserto.

5. Mostre que o mesmo método `calcularOrcamento()` se comporta de forma diferente para carros e motos, demonstrando **polimorfismo**.

5 Sistema de Pagamentos

Desenvolva um sistema para gerenciar diferentes formas de pagamento em uma loja virtual, utilizando herança e polimorfismo.

1. Crie uma **classe abstrata** chamada `Pagamento`. Ela deve ter os seguintes atributos:
   - `valor` – valor total da compra,
   - `data` – data do pagamento,

- `nomeCliente` – nome do cliente que realizou a compra.

A classe deve possuir:

- Um **método abstrato** chamado `processar()`, que realiza a lógica de processamento do pagamento.
- Um método `exibirRecibo()` que imprime as informações básicas da transação (nome do cliente, valor e data).

2. Crie as seguintes subclasses que herdem de `Pagamento`:

   (a) `CartaoCredito`:
   - Deve ter um atributo `parcelas` – quantidade de parcelas.
   - O método `processar()` deve calcular o valor final com juros de 2% por parcela e exibir o valor de cada parcela e o total atualizado.

   (b) `Boleto`:
   - Deve ter um atributo `desconto` – percentual de desconto.
   - O método `processar()` deve aplicar o desconto ao valor original e exibir o valor final.

   (c) `Pix`:
   - Não precisa de atributos adicionais.
   - O método `processar()` deve apenas confirmar o pagamento instantâneo, exibindo a mensagem de sucesso.

3. Crie uma lista contendo pelo menos:
   - Um pagamento via cartão de crédito,
   - Um pagamento via boleto,
   - Um pagamento via Pix.

4. Percorra a lista e, para cada pagamento:
   - Chame o método `exibirRecibo()` para mostrar os dados básicos da transação,
   - Em seguida, chame o método `processar()` para executar a lógica específica daquela forma de pagamento.

5. Demonstre **polimorfismo** mostrando que o mesmo método `processar()` se comporta de maneira diferente dependendo do tipo de pagamento.